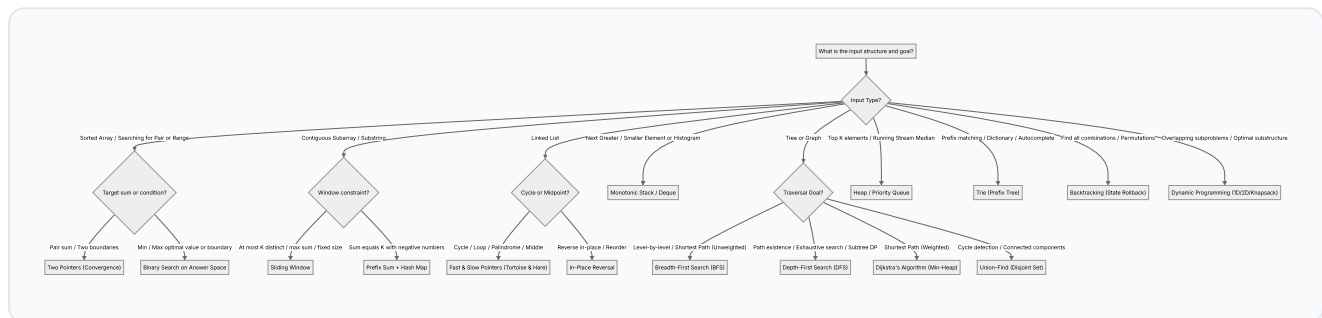


DSA Playbook Chapter 2: The 15 Essential Coding Patterns & Reusable Templates

Core Objective: Master the 15 fundamental algorithmic patterns that solve 95% of LeetCode Medium/Hard interview problems at Google, Meta, Amazon, and Microsoft.

0. The Master 1-Page Pattern Recognition Decision Tree

When you read an interview question, how do you instantly know which pattern to apply within 30 seconds? Follow this exact decision tree:



Rapid 30-Second Pattern Recognition Matrix

Clues in Problem Statement	Likely Optimal Pattern	Time Complexity
Sorted array, find pair summing to X	Two Pointers	$O(N)$ time, $O(1)$ space
"Longest substring with...", "Contiguous subarray with sum $\leq K$ "	Sliding Window	$O(N)$ time, $O(K)$ space
"Next greater temperature", "Largest rectangle in histogram"	Monotonic Stack	$O(N)$ time, $O(N)$ space
"Shortest path in unweighted grid", "Level-order traversal"	BFS with Queue	$O(V + E)$ time
"Count total paths", "Find maximum profit without adjacent items"	Dynamic Programming	$O(N)$ or $O(N \cdot M)$
"Top K most frequent words", "Merge K sorted streams"	Min/Max Heap	$O(N \log K)$
"Implement autocomplete", "Find words matching pattern with '.'"	Trie with DFS	$O(L)$ where L is word length

Pattern 1: Two Pointers (Convergence)

When to Use:

- Input array is **sorted**.
- Searching for pairs that meet a target condition without $O(N^2)$ nested loops.

Python Master Template:

Pattern 2: Sliding Window (Dynamic Expansion & Contraction)

When to Use:

- Searching for the **longest/shortest contiguous subarray or substring** satisfying a condition (e.g. at most K distinct characters).

Python Master Template:

Pattern 3: Fast & Slow Pointers (Floyd's Tortoise and Hare)

When to Use:

- Cycle detection in linked lists or arrays.
- Finding the exact midpoint of a linked list in a single pass.

Python Master Template:

Pattern 4: Monotonic Stack (Next Greater Element)

When to Use:

- Finding the **next greater** or **previous smaller** element for every position in an array in $O(N)$ time.

Python Master Template:

Pattern 5: Top K Elements (Min-Heap / Max-Heap)

When to Use:

- Finding the K largest or smallest elements in an unsorted list or streaming input without sorting the entire array in $O(N \log N)$.

Python Master Template:

Pattern 6: Binary Search on Answer (Monotonic Predicate)

When to Use:

- Optimization problems: *"Find the minimum speed to eat bananas in H hours"* or *"Find maximum capacity"*.

Python Master Template:



Pattern 7: Topological Sort (Kahn's In-Degree Algorithm)

When to Use:

- Task scheduling, course prerequisites, build systems represented as a Directed Acyclic Graph (DAG).

Python Master Template:



Further Reading

- [CP-Algorithms](#)
 - [VisuAlgo](#)
 - [Wikipedia: Dynamic programming](#)
-

Check Yourself

Answer in your head or on paper first, then open each answer.

▼ 1. Sorted array and a target sum: which pattern?

Two pointers from both ends, $O(n)$ time, $O(1)$ space.

▼ 2. Subarray or substring with a constraint on contiguous elements?

Sliding window.

▼ 3. Shortest path in an unweighted graph?

BFS.

▼ **4.** Overlapping subproblems with optimal substructure?

Dynamic programming (memoisation or tabulation).